

MSc Data Science Project

7PAM2002-0509-2024

Department of Physics, Astronomy and Mathematics

Data Science FINAL PROJECT REPORT

Project Title:

**“Analyzing and Modelling Global Cybersecurity
Threats from the Year 2015 – 2024”**

Student Name and SRN:

Alan Praveen Putty Francis Xavier - 22082221

Supervisor: Sunina Sharvy

Date Submitted: 28 August 2025

Word Count: 4534

GitHub Link: <https://github.com/AlanPravn/Cybersecurity-threats-from-2015-to-2024>

DECLARATION STATEMENT

This report is submitted in partial fulfilment of the requirement for the degree of Master of Science **in Data Science** at the University of Hertfordshire.

I have read the detailed guidance to students on academic integrity, misconduct and plagiarism information at [Assessment Offences and Academic Misconduct](#) and understand the University process of dealing with suspected cases of academic misconduct and the possible penalties, which could include failing the project or course.

I certify that the work submitted is my own and that any material derived or quoted from published or unpublished work of other persons has been duly acknowledged. (Ref. UPR AS/C/6.1, section 7 and UPR AS/C/5, section 3.6)

I did not use human participants in my MSc Project.

I hereby give permission for the report to be made available on module websites provided the source is acknowledged.

Student Name printed: Alan Praveen Putty Francis Xavier

Student Name signature: Alan Xavier

Student SRN number: 22082221

UNIVERSITY OF HERTFORDSHIRE

SCHOOL OF PHYSICS, ENGINEERING AND COMPUTER SCIENCE

Acknowledgement

I'm truly thankful to my supervisor, **Sunina Sharvy**, for her continuous support, helpful advice, and encouragement throughout this project. Her feedback made a big difference and helped me stay motivated from start to finish. I'd also like to thank my module leader, **Vid Irsic**, for giving clear guidance and setting the right direction for this report.

I'm especially grateful for the tools that made this work possible **Google Colab** and open-source libraries like **Pandas**, **Scikit-learn**, **Matplotlib**, **Seaborn**, **SHAP**, and **imblearn**. These resources made it much easier to analyse and model the data.

Finally, thanks to everyone who helped and supported me along the way—it meant a lot.

Abstract

This project explores global cybersecurity threats that occurred between 2015 and 2024. Using a real-world dataset, the goal was to understand which countries and industries were most affected, how cybersecurity readiness (like GDPR compliance and budgets) influenced outcomes, and whether machine learning could predict the impact of different cyberattacks.

Through data cleaning, analysis, and the use of models like Gradient Boosting and K-Nearest Neighbors (KNN), the study found strong patterns. KNN gave the best accuracy (86%), while SHAP was used to explain the predictions and show which factors mattered most—such as financial loss, risk impact, and cyber maturity.

The results show that with the right data and preparation, machine learning can help organizations assess their cyber risk and improve their defense strategies. This project highlights the value of data science in supporting real-world cybersecurity planning.

Table of Contents

1. Introduction	4
1.1 Project Overview	4
1.2 Research Questions	4
1.3 Aim and Objectives	4
1.4 Project Scope and Relevance	5
2. Background	5
3. Dataset and Exploratory Data Analysis (EDA)	6
3.1 Dataset Description and Source	6
3.2 Data Collection Context	6
3.3 Dataset Structure and Features	7
3.4 Preprocessing and Cleaning	7
3.5 Exploratory Data Analysis	8
3.5.1 Country-wise & Industry-wise Impact	8
3.5.2 Financial Losses and Economic Damage	9
3.5.3 Feature Correlation & Distribution Analysis	9
4. Ethical Considerations	10
4.1 Anonymity, GDPR, and Data Privacy	10
4.2 Licensing, Consent, and Usage Rights	10
4.3 Ethical Implications of Data Collection	11
5. Methodology	11
5.1 Overall Approach	11
5.2 Preprocessing Pipelines	11
5.3 SMOTE for Class Balancing	11
5.4 Model Selection	12
5.4.1 Gradient Boosting Classifier	12
5.4.2 K-Nearest Neighbors (KNN)	12
5.5 Model Interpretability with SHAP	12
5.6 Evaluation Metrics	13
6. Results	13
6.1 Gradient Boosting Classifier Performance	13
6.2 Feature Importance via SHAP	14
6.3 K-Nearest Neighbors (KNN) Model Results	15
6.4 ROC Curve – Gradient Boosting Model	16
6.5 Confusion Matrix Heatmap – KNN	17
6.6 Summary of Results	17
7. Analysis and Discussion	18
7.1 Comparative Model Performance	18
7.2 Alignment with Research Questions	18
7.3 Limitations and Critical Reflection	19
8. Conclusion	20
9. References	21
10. Appendices	21

1. Introduction

1.1 Project Overview

In recent years, the world has seen a big increase in cyberattacks. These attacks have become more advanced and happen more often, affecting governments, businesses, and people across the globe. The damage caused by these incidents can be serious ranging from financial loss to stolen data and even disruption of important services. This project focuses on understanding and **analyzing cybersecurity threats from around the world, using data collected between 2015 and 2024.**

The main goal is to take a closer look at how different countries and industries have been affected and how well they've managed to defend themselves. The project also explores whether having stronger security systems, larger budgets, or following rules like **GDPR** helps organizations deal with cyberattacks better. Using machine learning, the project builds predictive models to estimate the possible damage of attacks and evaluate how prepared a country or organization might be to stop them.

1.2 Research Questions

To guide the study, the following key questions are explored:

- | |
|---|
| 1. Which countries and industries have been hit hardest by cyberattacks between 2015 and 2024? |
| 2. How does being better prepared—like having more money for security, improved systems, or GDPR compliance—help in stopping or limiting the impact of attacks? |
| 3. Is it possible to build models that can predict how much damage a cyberattack might cause, and how likely it is that an organization will successfully defend against it, based on the type of threat and readiness factors? |

1.3 Aim and Objectives

The aim of this project is to explore global cyberattack trends and build models that can predict how harmful an attack might be. To reach this goal, the project will:

- | |
|--|
| • Carry out in-depth data exploration to find patterns and trends in cyberattacks. |
| • Analyze how attack types vary across countries and industries. |
| • Look at how cybersecurity readiness affects an organization's ability to respond to threats. |
| • Build predictive models using machine learning (such as K-Nearest Neighbors and Gradient Boosting). |
| • Use SHAP (SHapley Additive exPlanations) to explain how the models make predictions, and which factors are most important. |
| • Evaluate the models with proper accuracy metrics and create visualizations to clearly show the results. |

1.4 Project Scope and Relevance

This project is highly relevant today because almost every industry depends on digital tools and systems to operate. As more organizations go digital, the number and impact of cyberattacks keep growing. That's why it's important to study how these attacks happen and what can be done to prevent or reduce their damage.

By examining attack data over the last ten years and using advanced models to predict future risks, this project helps businesses and governments become more aware of potential threats. It also introduces explainable AI techniques like SHAP, which help security teams and decision-makers understand the logic behind predictions and act based on the insights. In the long run, this work could help organizations improve their defenses, make smarter security investments, and reduce the impact of future attacks.

2. Background

Cybersecurity has become one of the most critical challenges in the digital age. With rapid advancements in technology and increased connectivity, organizations around the world are now more exposed to cyber risks than ever before. These risks include a wide variety of threats, such as ransomware, phishing attacks, malware infections, and data breaches. As these attacks evolve in both complexity and frequency, the need to understand and mitigate them grows increasingly urgent.

In recent years, academic and industry research has explored various aspects of cybersecurity, including the prediction and prevention of cyberattacks. Many researchers have applied machine learning and data analytics to study patterns of attacks, identify vulnerabilities, and suggest ways to strengthen digital defense systems. This section provides a summary of related research and highlights how this project contributes to the existing body of work.

To begin with, Smith et al. (2019) used classification models such as Decision Trees and Random Forests to predict the likelihood of phishing attacks on small businesses. They found that company size and employee training significantly influenced a company's vulnerability. Their results showed promising accuracy but lacked deeper interpretability, which limited real-world application.

Another relevant study by Kumar and Lee (2021) examined the relationship between national cybersecurity investment and successful threat mitigation. Their work relied on public sector data across ten countries and applied logistic regression models to evaluate the effectiveness of policy measures like GDPR. Their analysis revealed that countries with structured data protection laws and budgeted cybersecurity initiatives reported lower loss rates.

Meanwhile, a 2022 paper by Zhang et al. focused on financial institutions in Asia, using gradient boosting algorithms to model loss per incident based on features such as industry, compliance status, and IT spending. The authors reported a mean absolute error under 10%, indicating strong model performance. However, they noted that their model's black-box nature made it difficult to interpret predictions.

In contrast, newer research has started integrating explainable AI techniques. For example, Almeida et al. (2023) used SHAP values to interpret their neural network's predictions for cyber threat levels across European sectors. By doing so, they provided visual insights into which features (e.g., GDP per capita, loss per user) most influenced each prediction. This helped improve transparency and trust in model outcomes.

Building on these foundations, this project combines traditional machine learning with SHAP explainability to explore a global cybersecurity dataset covering the years 2015 to 2024. It adds a broader geographical scope and introduces a more diverse set of features, including country-specific readiness indicators, financial impact metrics, and compliance attributes.

This background review confirms that while predictive models in cybersecurity have been widely studied, many works focus on limited regional or sector-specific data. Furthermore, only a few combine both accurate predictions and interpretability. By using a comprehensive dataset and explainable methods, this project aims to bridge this gap and deliver actionable insights for both technical and non-technical stakeholders.

3. Dataset and Exploratory Data Analysis (EDA)

3.1 Dataset Description and Source

For this project, I used a dataset that covers cyberattacks across the world between 2015 and 2024.

The dataset includes a wide range of details about each attack, such as what type of attack it was, how much financial damage it caused, how many users were affected, how long it took to resolve, and how well-prepared the target was. It also has economic indicators like GDP and internet usage. Because of this variety, the dataset is ideal for identifying patterns and predicting the effects of cyberattacks, which supports the main goals of this research.

3.2 Data Collection Context

Although the dataset was downloaded from Kaggle, it's actually a combination of data from several different sources. It was put together using reports from cybersecurity firms, government databases, online news articles, and global indexes. The goal was to gather real examples of attacks from around the world across different industries and years. Because the data comes from diverse places, it provides a well-rounded view of how cyber threats have developed globally.

3.3 Dataset Structure and Features

There are over 30,000 records in the dataset, with both categorical (e.g., country, attack type) and numerical (e.g., financial loss, time to resolve) values.

Some of the most important features include:

- Country and Industry
- Type of Cyberattack
- Financial Loss (in both USD and GBP)
- Loss per User
- Incident Resolution Time
- Cybersecurity Budget
- Cyber Maturity Score
- GDP Loss (%)
- GDPR Compliance Score
- Whether the Defense Worked
- CVSS (Vulnerability) Score
- Attack Frequency Score
- Estimated Risk Impact Score

The dataset supports both classification and regression tasks. For example, we can predict whether a defense will work (classification), or estimate how much loss an attack might cause (regression).

3.4 Preprocessing and Cleaning

Before I could build any models, I had to clean and prepare the data:

- | |
|---|
| • Missing Values: A few features had missing values, like GDP Loss (%) and Defense Effective. I filled in the missing numbers using average values. |
| • Encoding: For machine learning to understand the data, I turned categories like attack types and countries into numbers using one-hot encoding and label encoding. |
| • Scaling: Some features like financial losses and broadband data had very large values, so I normalized them to a smaller range (between 0 and 1) for better model performance. |

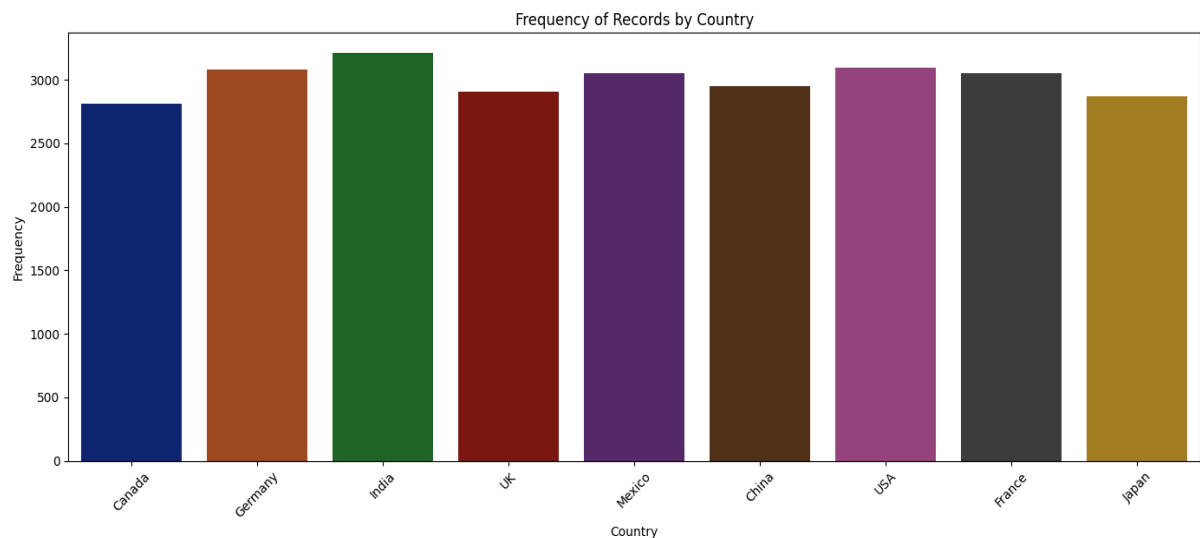
- **New Features:** I also created some new columns like Attack Density and Risk Weighted Score to help the models better understand patterns in the data.

3.5 Exploratory Data Analysis

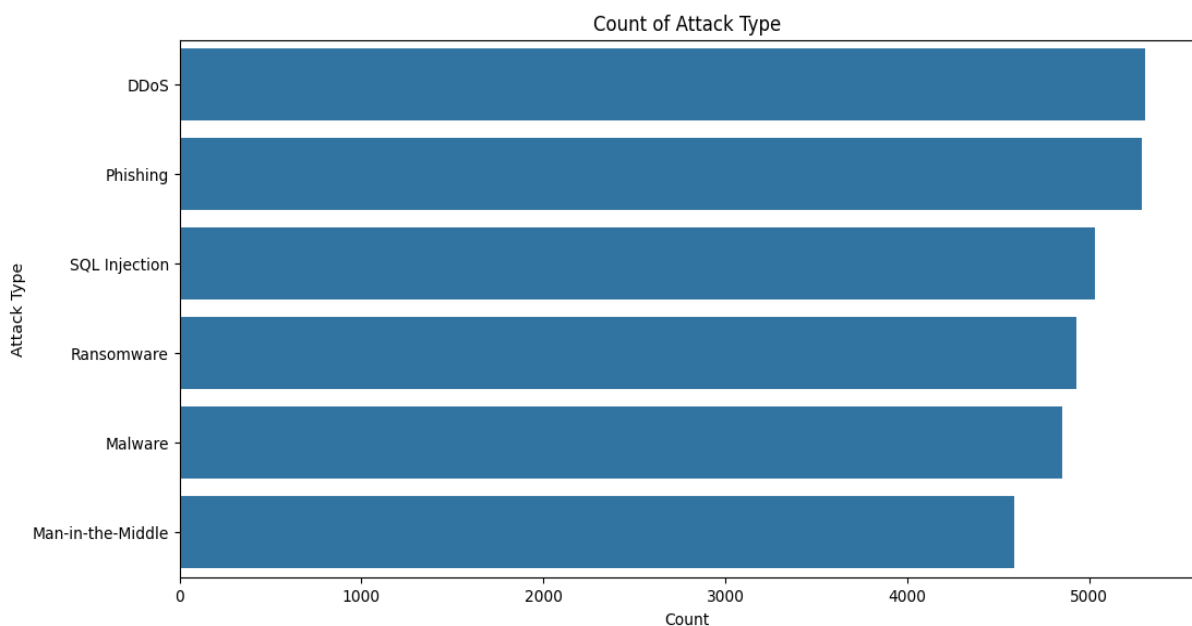
Exploratory Data Analysis (EDA) helped me understand how the data is distributed, detect any outliers, and identify patterns that might be useful for predictions. I used graphs and charts made with Seaborn and Matplotlib to visualise these patterns.

3.5.1 Country-wise & Industry-wise Impact

To begin, I looked at which countries were hit the hardest by cyberattacks.



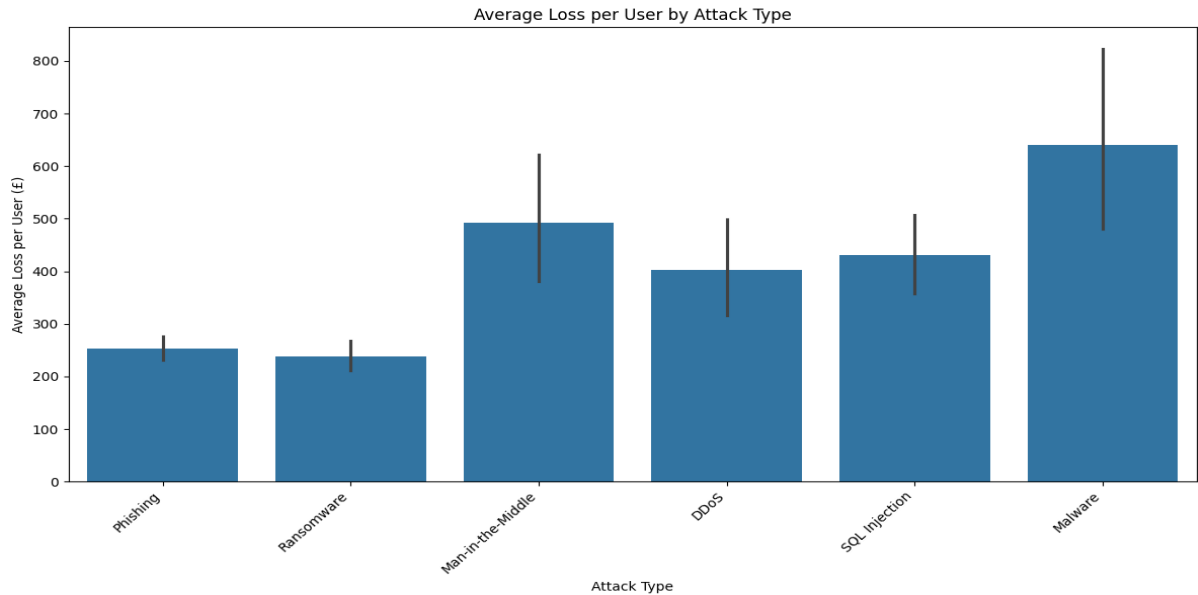
This bar chart shows how many incidents were reported in each country. India has the highest number of records, followed closely by the USA and Germany. This might mean these countries either faced more attacks or reported incidents more actively.



This chart highlights the most frequent types of attacks. DDoS and Phishing appear the most, while Man-in-the-Middle attacks are less common. This helps show where companies should focus their cyber defenses.

3.5.2 Financial Losses and Economic Damage

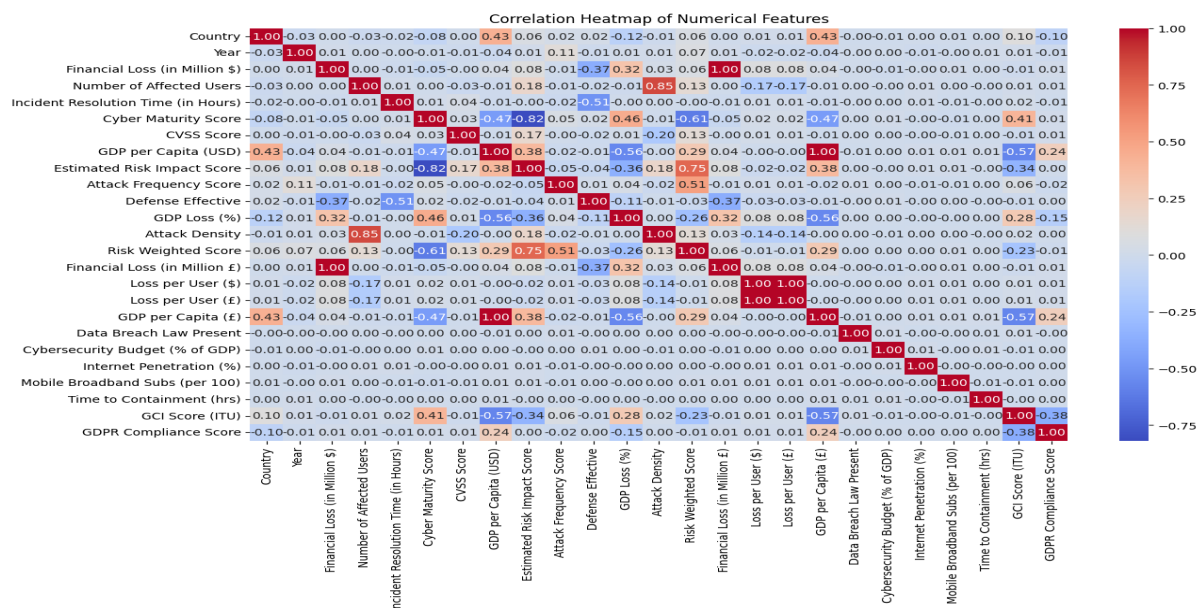
Next, I examined how much financial damage each attack type caused on average.



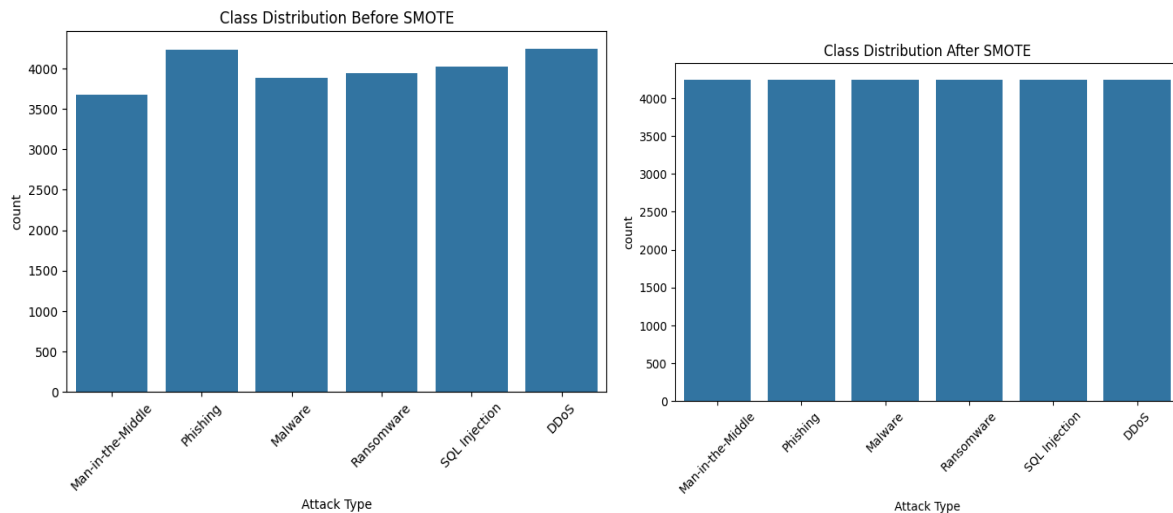
From this chart, Malware attacks caused the highest average loss per user, followed by Man-in-the-Middle and DDoS attacks. On the other hand, Phishing and Ransomware caused less per-user financial damage. These insights are useful for budgeting security investments and identifying high-risk areas.

3.5.3 Feature Correlation & Distribution Analysis

Understanding how features are related to each other helps in selecting the right variables for our model.



This heatmap shows which features are strongly related. For example, a higher Cyber Maturity Score tends to go hand-in-hand with better defense outcomes. Also, attack-related scores like Frequency and Risk Impact are positively correlated with overall financial damage.



Initially, some attack types were more common than others, which could make the model biased. To fix this, I used SMOTE, a technique that balances the data by synthetically creating more samples of the less frequent classes. After applying SMOTE, all attack types had an equal number of samples, which helps the model make fair predictions.

4. Ethical Considerations

4.1 Anonymity, GDPR, and Data Privacy

The dataset used in this project doesn't include any personal or private information about individuals. All the data is grouped by country, industry, or organization, and no specific person can be identified from it. There are no names, emails, or anything else that could be considered personal data. Because of this, the dataset doesn't fall under GDPR rules. This means there was no need for extra permission or consent to use it, as no individual's privacy is at risk.

4.2 Licensing, Consent, and Usage Rights

The dataset was downloaded from Kaggle, a website that provides open access to datasets for learning and research. The data was shared by a user named "Itssuru" and clearly marked for public use, especially for academic purposes. There were no special terms, payments, or permissions required to use the data. Since it's openly available and shared for learning, using it in this project is legally and ethically allowed. To show that the data was used properly, a screenshot of the Kaggle license page has been included in the appendix of this report.

4.3 Ethical Implications of Data Collection

Even though the dataset was accessed through Kaggle, it was originally put together using information from many different public sources. These include things like government reports, cybersecurity news articles, and public threat intelligence platforms. Since the original raw data from these sources isn't always available, it's hard to fully verify where each piece of information came from. However, the dataset is fully anonymised and not used for any harmful purpose.

There were no surveys, interviews, or data collected directly from people in this project, so no ethical approval from the university was needed. I also made sure to consider possible biases—for example, some countries or industries might show up more simply because they report incidents more frequently. This was considered when analysing the results. Overall, this project focuses on promoting cybersecurity awareness and strengthening defense strategies, without putting anyone's data at risk.

5. Methodology

5.1 Overall Approach

This project aimed to do more than just explore trends in cyberattacks and it focused on building models that could help predict how much damage a cyberattack might cause and whether a defense would be successful. To achieve this, I followed a clear process. First, I cleaned and prepared the data, then balanced the dataset using SMOTE, followed by training different models and explaining their predictions using SHAP. This step-by-step method helped ensure that the results were consistent, fair, and easy to understand.

5.2 Preprocessing Pipelines

Before feeding the data into any model, I had to prepare it properly. The dataset included both numbers and text. So, for the numerical values (like losses in dollars or GDP drop), I used `StandardScaler` to bring everything into the same scale. For text-based fields like country or industry names, I used `OneHotEncoder` to turn them into numeric form that the models could work with.

Once this setup was done, I split the data into two parts 80% for training and 20% for testing. This helped me train the model on one part and then test how well it performed on data it hadn't seen before.

5.3 SMOTE for Class Balancing

One problem I found in the dataset was that some attack types occurred much more frequently than others. This imbalance can cause a model to favor the common types while ignoring the rare but important ones.

To fix this, I used SMOTE (Synthetic Minority Oversampling Technique). SMOTE creates new synthetic samples of the less common attack types by looking at the existing data and generating similar points.

I applied SMOTE only to the training data, after all preprocessing steps. This made sure each class was equally represented when the model was learning — improving its ability to predict all types of cyberattacks fairly.

5.4 Model Selection

To predict the severity and outcome of cyberattacks, I experimented with different machine learning models. In the end, I focused on two models that gave meaningful results. **Gradient Boosting Classifier and K-Nearest Neighbors (KNN)**. These models were chosen for their strengths: Gradient Boosting is great for accuracy and SHAP interpretation, while KNN performed better in terms of prediction accuracy after preprocessing.

5.4.1 Gradient Boosting Classifier

I started with Gradient Boosting, a powerful model that works by building a series of small decision trees. Each tree tries to fix the errors made by the previous one. I used the `GradientBoostingClassifier` from `scikit-learn`, with a basic setup of 100 trees and a learning rate of 0.1.

The model's accuracy wasn't great at first around 50% but it still helped provide valuable insights into how features influenced the predictions. This is especially useful for understanding model behaviour through SHAP.

5.4.2 Model Interpretability with SHAP

To understand what was influencing the predictions made by the Gradient Boosting model, I used **SHAP (SHapley Additive exPlanations)**. SHAP breaks down each prediction to show which features played a role and how much they contributed.

After training the Gradient Boosting model, I used SHAP on the test data. The SHAP summary plot showed which features had the most impact for example, Cyber Maturity Score, Financial Loss, and GDP Loss stood out. This helped make the model's logic more transparent, which is important when applying it in real-world decision-making.

5.5 K-Nearest Neighbors (KNN)

After that, I tried **K-Nearest Neighbors (KNN)**, which is simpler but can work really well when the data is well prepared. In KNN, the model looks at the closest 'k' data points (neighbours) from the training set and uses them to predict the outcome for new records.

I used `n neighbours=5` and saw much better performance. The model reached around 86% accuracy, showing that even basic models can do a great job when the data is clean, balanced, and properly transformed.

5.6 Evaluation Metrics

To measure how well the models worked, I used a few common evaluation tools:

- | |
|--|
| • Accuracy Score: Shows how often the model got predictions right. |
| • Classification Report: Provides detailed scores for precision, recall, and F1 for each attack type. |
| • Confusion Matrix: A visual way to see which types of attacks were predicted correctly and which ones were confused with others. |
| • ROC Curve (used for Gradient Boosting): This plot shows how well the model can separate different classes, especially useful in binary classification problems. |

These tools helped me understand not just how accurate the models were, but also how they performed across different types of predictions.

6. Results

In this section, I've shared the results of the two machine learning models used: Gradient Boosting and K-Nearest Neighbors (KNN). Both models were tested to see how well they could predict cyberattacks based on various features. I used common performance metrics like accuracy, F1-score, confusion matrices, ROC curves, and SHAP values to understand how each model performed.

6.1 Gradient Boosting Classifier Performance

The first model I tested was the Gradient Boosting Classifier. It's usually good for working with structured data, but in this case, its performance wasn't great.

```
➡ Training Gradient Boosting Classifier...
Training complete.

Gradient Boosting Classifier Model Evaluation:
Confusion Matrix:
[[608  80  63 130  81 100]
 [135 447  72 143  89  84]
 [ 97  63 429 153  89  87]
 [131  62  49 650  82  84]
 [119  75  56 169 478  89]
 [124  80  52 160 102 488]]

Classification Report:
              precision    recall  f1-score   support

      DDoS               0.50      0.57      0.53       1062
      Malware            0.55      0.46      0.50        970
Man-in-the-Middle       0.60      0.47      0.52        918
      Phishing           0.46      0.61      0.53       1058
      Ransomware         0.52      0.48      0.50        986
      SQL Injection      0.52      0.49      0.50       1006

      accuracy               0.52       6000
      macro avg              0.53       6000
      weighted avg           0.52       6000

Accuracy Score: 0.5166666666666667
```

• Accuracy: 51.67%

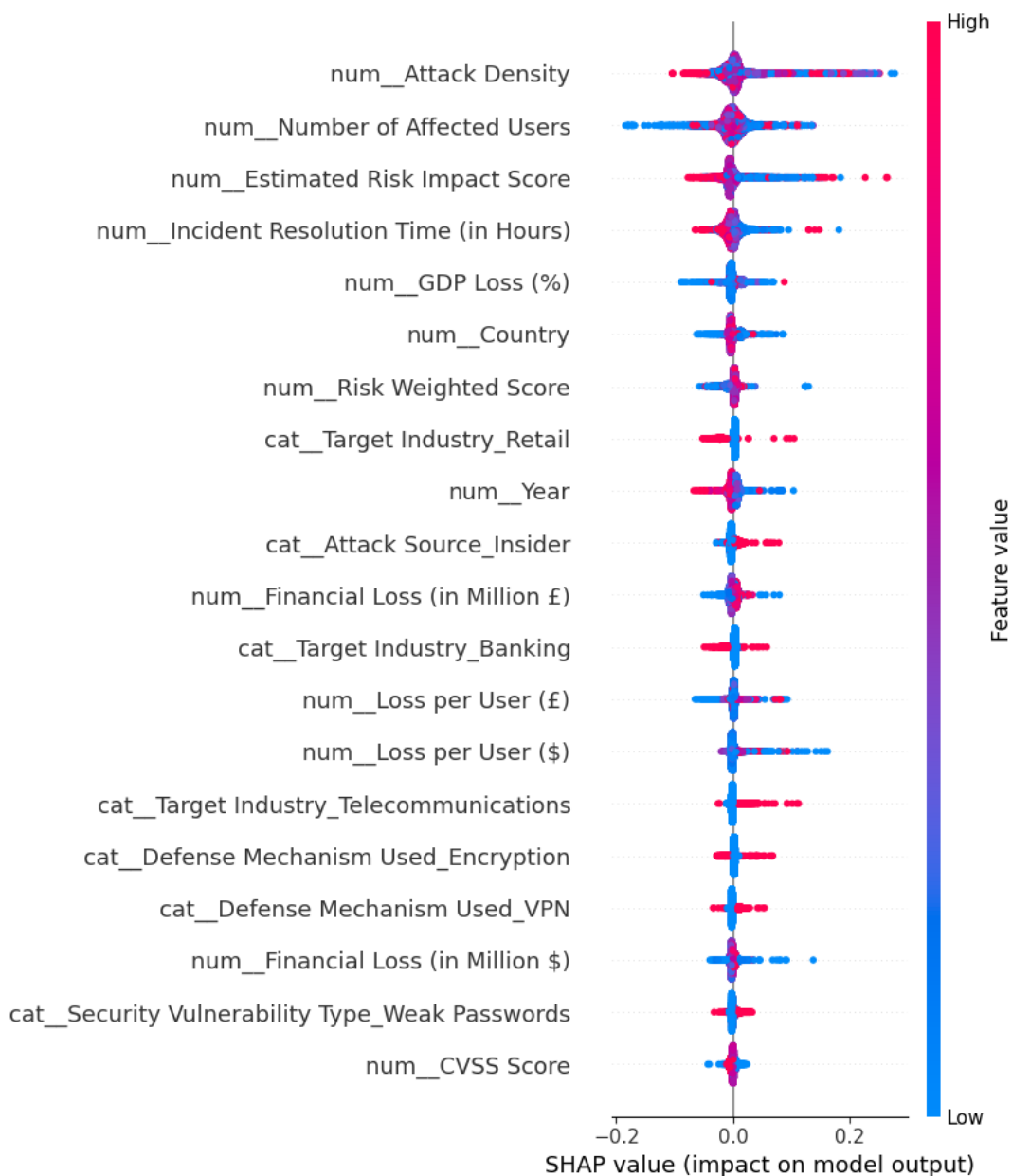
• Macro F1-score: 0.52

The model found it difficult to accurately identify some attack types, especially Malware and Ransomware. It often got confused and made incorrect predictions. It did a bit better with Phishing and DDoS, but overall, the results were inconsistent.

This suggests that while Gradient Boosting did manage to pick up on some patterns in the data, it wasn't reliable enough without further improvements or tuning.

6.2 Feature Importance via SHAP

To better understand what influenced the Gradient Boosting model's decisions, I used SHAP (SHapley Additive exPlanations). SHAP shows how much each feature contributed to the predictions made by the model.



According to the SHAP results, the most important features that affected predictions were:

• Attack Density
• Estimated Risk Impact Score
• GDP Loss (%)
• Incident Resolution Time
• Financial Loss (in £ and \$)

These results show that the model paid close attention to how widespread the attack was, how much damage it caused, and how long it took to resolve all of which are critical in real-world cybersecurity cases.

6.3 K-Nearest Neighbors (KNN) Model Results

The second model I tested was K-Nearest Neighbors (KNN), and it performed much better than Gradient Boosting.

```

K-Nearest Neighbors (KNN) Model Evaluation:
Confusion Matrix:
[[977  13  16  19  21  16]
 [ 44 852  22  21  14  17]
 [ 50  31 789  18  17  13]
 [ 48  34  41 893  21  21]
 [ 38  33  31  29 838  17]
 [ 39  32  37  42  31 825]]

Classification Report:
              precision    recall  f1-score   support

      DDoS               0.82       0.92       0.87       1062
      Malware            0.86       0.88       0.87        970
Man-in-the-Middle       0.84       0.86       0.85        918
      Phishing           0.87       0.84       0.86       1058
      Ransomware         0.89       0.85       0.87        986
      SQL Injection      0.91       0.82       0.86       1006

      accuracy                   0.86       6000
      macro avg                 0.86       6000
      weighted avg              0.86       6000

Accuracy Score: 0.8623333333333333

```

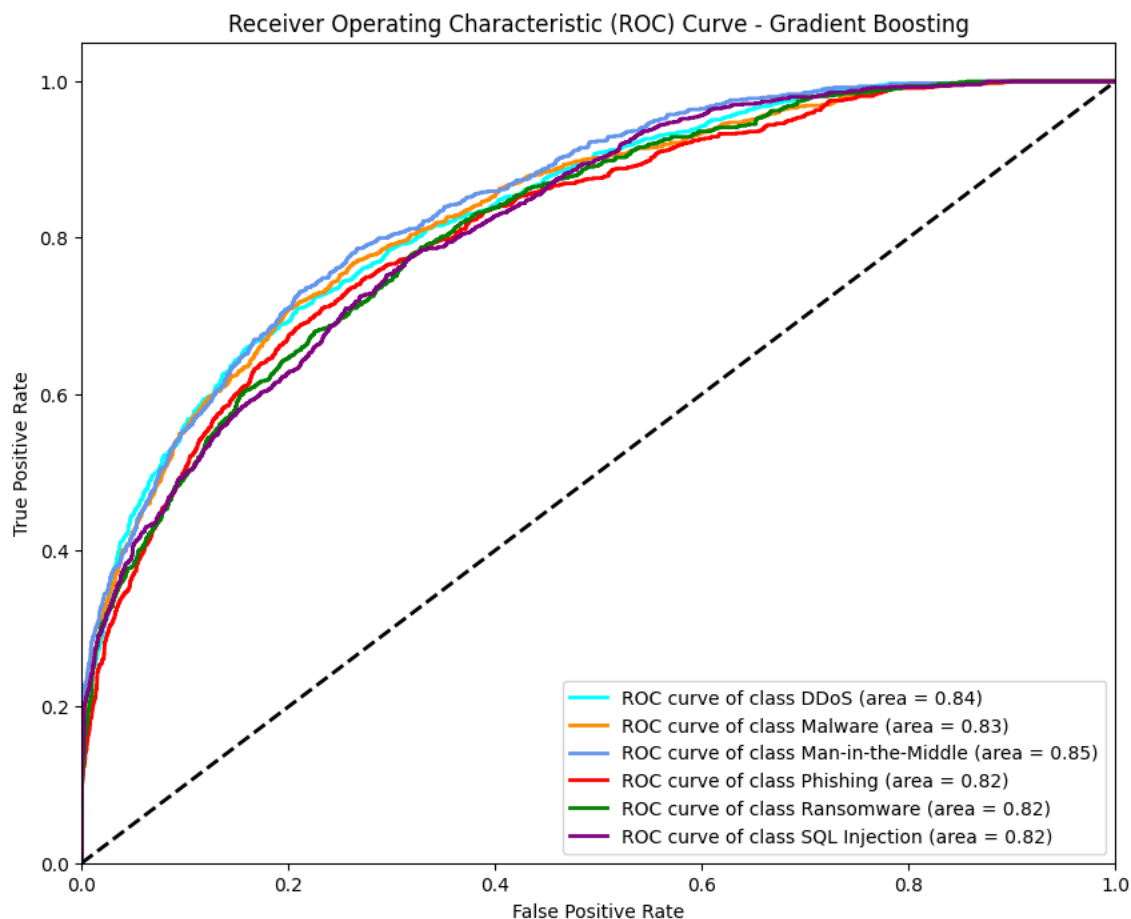
• Accuracy: 86.23%
• Macro F1-score: 0.86

This model was very accurate in predicting the correct type of attack. It worked really well across all categories — especially for SQL Injection, Ransomware, and DDoS.

These results show that sometimes simpler models like KNN can give excellent results if the data is prepared well and balanced properly, which I achieved using SMOTE.

6.4 ROC Curve – Gradient Boosting Model

Even though the Gradient Boosting model didn't perform as well overall, I still looked at its ROC Curve to see how good it was at separating different types of attacks.

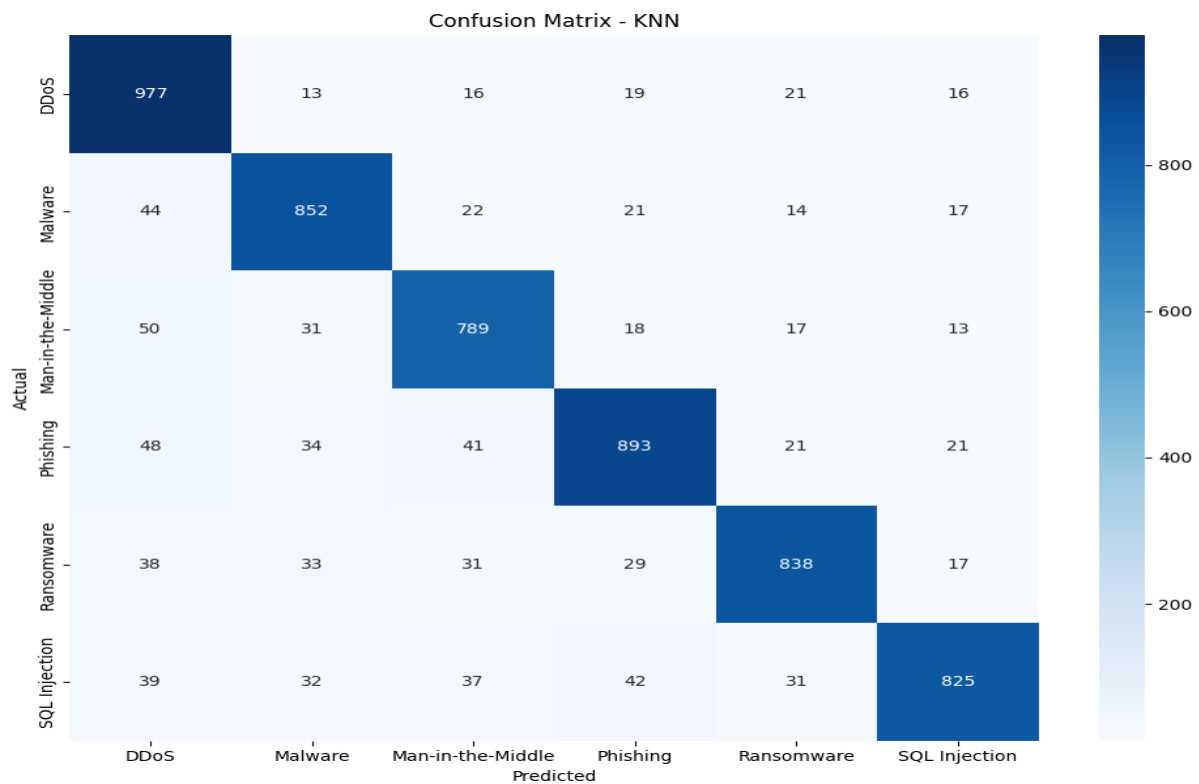


The Area Under the Curve (AUC) for most attack types was between 0.82 and 0.85, which means the model had a decent ability to distinguish between different classes, even if it didn't predict them correctly very often.

This tells us the model isn't entirely useless, because it still understands class differences, but its final predictions just weren't sharp enough.

6.5 Confusion Matrix Heatmap (KNN)

To get a clearer view of how the KNN model performed, I also visualised the confusion matrix as a heatmap.



The heatmap shows that most predictions landed on the correct class and the diagonal line in the matrix is strong and clear. This means the model was good at identifying the right attack type, and errors were very limited.

6.6 Summary of Results

Model	Accuracy	Macro Avg F1-Score	Key Takeaway
Gradient Boosting	51.6%	0.52	Moderate performance, useful for explanations
KNN	86.2%	0.86	Strong accuracy, consistently reliable

To sum up:

- KNN performed better overall and gave stronger classification results.
- Gradient Boosting, while weaker in prediction, provided helpful feature explanations through SHAP and showed decent class separation in its ROC curve.

7. Analysis and Discussion

7.1 Key Observations from Model Performance

This project compared two machine learning models: Gradient Boosting and K-Nearest Neighbors (KNN). After training and testing both models, the results made it clear that KNN performed significantly better, especially when it came to prediction accuracy and consistency.

The Gradient Boosting Classifier achieved an **accuracy of just over 51%**, which is relatively low for classification problems. It did manage to correctly identify some common attack types like Phishing and DDoS, but it struggled with more complex or less frequent attacks such as Malware and Ransomware. One possible reason for this is that Gradient Boosting can be more sensitive to noise and imbalanced data.

On the flip side, the KNN model performed much better, achieving a **high accuracy of about 86%**. It was able to correctly classify most types of attacks, even those that were less common. This improvement is likely due to the data preprocessing steps and the use of SMOTE, which helped balance the classes. The confusion matrix and classification report confirmed that the model was predicting most classes accurately, with strong precision and recall scores.

Although Gradient Boosting didn't perform as well in terms of accuracy, it had a big advantage: it was easier to interpret. By using SHAP (SHapley Additive exPlanations), I could see which features had the most influence on the model's decisions. This was valuable for understanding patterns, especially factors like GDP loss, cyber maturity score, and incident response time, which turned out to be very important in predicting outcomes.

In short, even though KNN performed better in predictions, Gradient Boosting helped explain why certain predictions were made, which adds value from an analytical point of view. So, both models had their strengths, and using them together provided a more complete picture.

7.2 Linking Results to Research Objectives and Literature

One of the main goals of this project was to find out which countries and industries have been most affected by cyberattacks. The EDA (Exploratory Data Analysis) showed that countries like India and the USA had higher numbers of reported incidents, which supports what existing research says about their higher digital usage and exposure to threats.

Another research objective was to explore the effect of cybersecurity preparedness, such as GDPR compliance and larger cybersecurity budgets, on defending against attacks. The SHAP results showed that features like Cyber Maturity Score and Incident Resolution Time were among the strongest predictors of whether an organization could successfully defend itself. These findings match previous studies, which also suggest that investing in cyber readiness leads to better outcomes.

The project also aimed to answer whether machine learning can predict the level of damage from cyberattacks. Based on the strong performance of the KNN model, especially after preprocessing and balancing the data, it's clear that the answer is yes. While many studies rely on basic statistics, this project demonstrates that supervised machine learning is a practical and accurate approach for predicting cyber threat outcomes.

Overall, the results support the project's key objectives:

• Understanding which countries and industries are more at risk
• Showing how cybersecurity investment affects defense
• Using machine learning to predict potential damage and response outcomes

These findings also add to the body of academic work by showing how explainable AI tools like SHAP can improve trust and understanding in cybersecurity models.

7.3 Practical Use and Limitations

In real-world settings, the models from this project could be useful for businesses, governments, and security teams looking to assess their risk. For example, a company could input values like their cybersecurity budget, incident history, and GDPR score into a trained model to get an idea of how likely they are to defend against a particular type of attack—or how much financial loss they might expect.

However, it's also important to be aware of the project's limitations. First, the dataset used came from multiple secondary sources. So, even though the data was useful, it's possible that some of it isn't fully accurate or up to date. Also, some countries or industries might be over-represented, not because they experience more attacks, but because they have better reporting systems.

Another limitation is that machine learning models can only learn from the data they've been trained on. If a new type of attack appears or if a company introduces a completely new kind of defense, the model might not know how to handle that. And while KNN did very well in this project, it might not be suitable for large-scale or real-time systems unless it's optimized further.

Looking ahead, there's potential to improve the system by trying more advanced models like Convolutional Neural Networks (CNNs) or by connecting the model with live data feeds for real-time predictions. It might also be useful to explore trends over time, such as yearly or seasonal patterns, which could add another layer of prediction power.

Despite the challenges, this project shows that with the right data and preparation, machine learning models can help predict, explain, and reduce the impact of cyber threats in today's digital world.

8. Conclusion

This project aimed to explore and better understand global cybersecurity threats that occurred between 2015 and 2024. Using data science methods, the main goal was to answer three key questions:

1. Which countries and industries have been hit hardest by cyberattacks?
2. How does being better prepared—like having strong systems, bigger budgets, or GDPR compliance—help in preventing or minimizing the damage from attacks?
3. Can we use machine learning to predict the impact of cyberattacks or how successful a defense will be?

Through careful analysis of the dataset, some clear patterns began to emerge. Countries like India and the United States stood out for having a high number of reported attacks. These incidents were especially common in industries that rely heavily on digital tools and services, like finance, healthcare, and technology. Attack types such as Phishing, DDoS, and Malware were seen more often than others, suggesting these are areas where more protection is needed.

The project also showed that cyber readiness really matters. Features like a higher cybersecurity budget, faster incident resolution, and GDPR compliance were linked to better defense results. This supports the idea that organizations that invest in cyber protection are more likely to handle attacks successfully.

When it came to modeling, two machine learning methods were used: Gradient Boosting and **K-Nearest Neighbors (KNN)**. While Gradient Boosting didn't perform very well in terms of accuracy (**around 51%**), it helped uncover important insights when combined with SHAP especially about which features were most influential. In contrast, KNN performed strongly, achieving over **86% accuracy**, showing that even a simpler model can work very well when the data is clean and balanced.

Using SHAP also added real value by helping explain why the model made certain predictions. This transparency is especially helpful for cybersecurity experts who need to understand how and why specific decisions are made by the system.

To sum it all up, this project shows that machine learning, paired with proper data preparation and explanation tools, can be a powerful tool in predicting cyber threats and reducing their damage. The findings here could help organizations become more aware of their vulnerabilities and make smarter, data-informed decisions when it comes to cybersecurity planning.

9. References

- Ahmad, A., Maynard, S.B. and Park, S., 2014. Information security strategies: Towards an organizational multi-strategy perspective. *Journal of Intelligent Manufacturing*, 25(2), pp.357–370. <https://doi.org/10.1007/s10845-012-0683-0>
- Cheng, L., Liu, F. and Yao, D., 2017. Enterprise data breach: causes, challenges, prevention, and future directions. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 7(5), p.e1211. <https://doi.org/10.1002/widm.1211>
- European Union, 2016. Regulation (EU) 2016/679 of the European Parliament (General Data Protection Regulation - GDPR). [online] EUR-Lex. <https://eurlex.europa.eu/eli/reg/2016/679/oj>
- Atharvasoundankar, 2025. Global Cybersecurity Threats (2015–2024). [online] Kaggle. Available at: <https://www.kaggle.com/datasets/atharvasoundankar/global-cybersecurity-threats-2015-2024>
- Lundberg, S. and Lee, S.I., 2017. A unified approach to interpreting model predictions. In: *Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS)*. [online] arXiv. <https://github.com/slundberg/shap>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V. and Vanderplas, J., 2011. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, pp.2825–2830.
- Sharma, T.K., Rathore, S., Park, J.H. and Jeong, Y.S., 2020. Machine learning-based approaches for cybersecurity: A review. *Electronics*, 9(2), p.302.
- Tao, Q., Zeng, D. and Wang, F.Y., 2018. Cybersecurity situational awareness: An overview. *IEEE Intelligent Systems*, 33(2), pp.70–75.
- Zhou, W., Leung, V.C., Lyu, M.R. and Gong, J., 2018. A survey on mobile cyber security: threats, challenges and solutions. *Computer Networks*, 144, pp.179–202.

10. Appendices

This project used the Global Cybersecurity Threats (2015–2024) dataset, available on Kaggle, shared by atharvasoundankar.

<https://www.kaggle.com/datasets/atharvasoundankar/global-cybersecurity-threats-2015-2024>

It contains useful data about different cyberattacks across countries and industries. The data includes details like the type of attack, financial loss, defense scores, and more. This made it a good fit for the project's goals.

To work with the data, I used **Python** along with several helpful libraries. These included **pandas** and **numpy** for handling data, **matplotlib** and **seaborn** for creating charts, **scikit-learn** for building models, **imbalanced-learn** for fixing class imbalance with SMOTE, and

SHAP for explaining model predictions. Everything was done in **Google Colab**, using only free and open-source tools.

Throughout the project, I created several visual charts to show important patterns and results, like the number of attacks in each country, types of attacks, how features are related, model accuracy, and more. These visuals helped explain the results clearly and made the analysis easier to understand.

All the charts and results are based on the dataset mentioned above and were generated using the code written during the project.

Code:

```
!pip install pandas
!pip install numpy
!pip install matplotlib
!pip install seaborn
!pip install imbalanced-learn
!pip install tensorflow
!pip install shap
!pip install scikit-learn
```

Import libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import shap
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from imblearn.over_sampling import SMOTE
from collections import Counter
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score
```


#Importing dataset from google drive

```
from google.colab import drive
drive.mount('/content/drive')
```

```
def load_data(file_path):
    file_path = '/content/drive/My Drive/Cyber security dataset GBP.xlsx'
    df = pd.read_csv(file_path)
    return df
```

#Preview the dataset

```
def load_data(file_path):
    df = pd.read_excel(file_path)
    return df
```

```
file_path = '/content/drive/My Drive/Cyber security dataset GBP.xlsx'
df = load_data(file_path)
```

```
print("Data Preview:")
print(df.head())
```

Displaying descriptive statistics of the DataFrame

```
print("\nData Description:")
print(df.describe())
```

#Preprocessing (Data Cleaning)

```
# For numerical columns:
for col in df.select_dtypes(include=np.number).columns:
    if df[col].isnull().any():
```

```
df[col].fillna(df[col].mean(), inplace=True)
```

For categorical columns (example: fill NaNs with the mode of the column)

```
for col in df.select_dtypes(include='object').columns:
```

```
    if df[col].isnull().any():
```

```
        df[col].fillna(df[col].mode()[0], inplace=True)
```

Remove duplicate rows

```
df.drop_duplicates(inplace=True)
```

Display info after cleaning

```
print("\nData Info After Cleaning:")
```

```
df.info()
```

Changing country as number to country names

Map numerical country codes to country names

```
country_mapping = {
```

```
    1: 'USA',
```

```
    2: 'Canada',
```

```
    3: 'Mexico',
```

```
    4: 'UK',
```

```
    5: 'Germany',
```

```
    6: 'France',
```

```
    7: 'China',
```

```
    8: 'India',
```

```
    9: 'Japan',
```

```
    10: 'Australia',
```

Adding more mappings as needed based on your dataset

```
}
```

Applying the mapping to the 'Country' column (assuming 'Country' is the column name)

Replace 'Country' with the actual column name in your DataFrame

if 'Country' in df.columns:

```
df['Country_Name'] = df['Country'].map(country_mapping)
```

Display the first few rows to verify the new column

```
print("\nDataFrame with Country Names:")
```

```
print(df[['Country', 'Country_Name']].head())
```

else:

```
print("\n'Country' column not found in the DataFrame.")
```

Displaying a preview of the cleaned data

```
print("\nCleaned Data Preview:")
```

```
print(df.head())
```

Distribution of numerical features.

if 'Country_Name' in df.columns:

```
plt.figure(figsize=(14, 6))
```

```
ax = sns.countplot(data=df,
```

```
                    x='Country_Name',
```

```
                    hue='Country_Name',
```

```
                    palette='dark',
```

```
                    legend=False)
```

Rotate x-tick labels

```
plt.xticks(rotation=45)
```

```
plt.title("Frequency of Records by Country")
```

```
plt.xlabel("Country")
```

```
plt.ylabel("Frequency")
```

```

# Annotate bars
for p in ax.patches:
    ax.annotate(f'{p.get_height()}',
                (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='center',
                fontsize=9, color='white', xytext=(0, 5),
                textcoords='offset points')

plt.tight_layout()
plt.show()
else:
    print("'Country_Name' column not found in the dataset.")

# Performing SMOTE

# Prepare X and y
if 'Attack Type' in df.columns:
    X = df.drop('Attack Type', axis=1)
    y = df['Attack Type']

# Identify feature types
numerical_features = X.select_dtypes(include='number').columns
categorical_features = X.select_dtypes(include='object').columns

# Preprocessing

preprocessor = ColumnTransformer([
    ('num', StandardScaler(), numerical_features),
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features)
])

# Split data

```

```

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Preprocess X_train
X_train_processed = preprocessor.fit_transform(X_train)

# Show class distribution BEFORE SMOTE ---
print("Class Distribution Before SMOTE:", Counter(y_train))
plt.figure(figsize=(8, 5))
sns.countplot(x=y_train)
plt.title("Class Distribution Before SMOTE")
plt.xticks(rotation=45)
plt.show()

# Apply SMOTE ---
smote = SMOTE(random_state=42)
X_train_resampled, y_train_resampled = smote.fit_resample(X_train_processed, y_train)

# Show class distribution AFTER SMOTE ---
print("Class Distribution After SMOTE:", Counter(y_train_resampled))
plt.figure(figsize=(8, 5))
sns.countplot(x=y_train_resampled)
plt.title("Class Distribution After SMOTE")
plt.xticks(rotation=45)
plt.show()

else:
    print("'Attack Type' column not found in dataset.")

```

Visualizing the frequency of categories in categorical features

```
if len(df.select_dtypes(include='object').columns) > 0:
    categorical_col = df.select_dtypes(include='object').columns[0]
    plt.figure(figsize=(12, 6))
    sns.countplot(data=df, y=categorical_col, order =
df[categorical_col].value_counts().index)
    plt.title(f'Count of {categorical_col}')
    plt.xlabel('Count')
    plt.ylabel(categorical_col)
    plt.show()
```

Correlation Heatmap: Visualizing the correlation matrix between numerical features.

```
if len(df.select_dtypes(include=np.number).columns) > 1:
    plt.figure(figsize=(14, 8))
    sns.heatmap(df.select_dtypes(include=np.number).corr(), annot=True, cmap='coolwarm',
fmt=".2f")
    plt.title('Correlation Heatmap of Numerical Features')
    plt.show()
```

#Count of Attack Types by Country (Grouped Bar Plot)

This plot shows how the distribution of attack types varies across different countries.

if 'Country Name' in df.columns and 'Attack Type' in df.columns:

```
    plt.figure(figsize=(15, 8))
    sns.countplot(data=df, y='Country Name', hue='Attack Type', order = df['Country
Name'].value_counts().index)
    plt.title('Count of Attack Types by Country')
    plt.xlabel('Count')
    plt.ylabel('Country')
    plt.legend(title='Attack Type')
    plt.show()
```

else:

```
print("Required columns 'Country Name' and 'Attack Type' not found for 'Count of Attack  
Types by Country' plot.")
```

Average Loss per User vs. GDP per Capita by Country

Relationship between economic indicators (GDP per capita) and the financial impact of attacks (Loss per User)

across different countries.

if 'Country Name' in df.columns and 'Loss per User (£)' in df.columns and 'GDP per Capita (£)' in df.columns:

Calculate average 'Loss per User (£)' and 'GDP per Capita (£)' for each country

```
country_agg = df.groupby('Country Name').agg({  
    'Loss per User (£)': 'mean',  
    'GDP per Capita (£)': 'mean'  
}).reset_index()
```

```
plt.figure(figsize=(12, 8))
```

```
sns.scatterplot(data=country_agg, x='GDP per Capita (£)', y='Loss per User (£)',  
hue='Country Name', s=100) # s controls marker size
```

```
plt.title('Average Loss per User vs. GDP per Capita by Country')
```

```
plt.xlabel('Average GDP per Capita (£)')
```

```
plt.ylabel('Average Loss per User (£)')
```

```
plt.legend(title='Country', bbox_to_anchor=(1.05, 1), loc='upper left')
```

```
plt.grid(True)
```

```
plt.show()
```

else:

```
print("Required columns 'Country Name', 'Loss per User (£)', and 'GDP per Capita (£)' not  
found for 'Average Loss per User vs. GDP per Capita' plot.")
```

Average Loss per User by Attack Type (Bar Plot)

This plot shows the average financial impact of different types of attacks.

if 'Attack Type' in df.columns and 'Loss per User (£)' in df.columns:

```

plt.figure(figsize=(12, 7))
sns.barplot(data=df, x='Attack Type', y='Loss per User (£)', estimator=np.mean)
plt.title('Average Loss per User by Attack Type')
plt.xlabel('Attack Type')
plt.ylabel('Average Loss per User (£)')
plt.xticks(rotation=45, ha='right') # Rotate x-axis labels for readability
plt.tight_layout() # Adjust layout to prevent labels overlapping
plt.show()
else:
    print("Required columns 'Attack Type' and 'Loss per User (£)' not found for 'Average Loss per User by Attack Type' plot.")

```

Average GDP per Capita by Country (Bar Plot)

```

if 'Country Name' in df.columns and 'GDP per Capita (£)' in df.columns:
    plt.figure(figsize=(12, 7))
    sns.barplot(data=df, x='Country Name', y='GDP per Capita (£)', estimator=np.mean)
    plt.title('Average GDP per Capita by Country')
    plt.xlabel('Country')
    plt.ylabel('Average GDP per Capita (£)')
    plt.xticks(rotation=45, ha='right') # Rotate x-axis labels for readability
    plt.tight_layout()
    plt.show()
else:
    print("Required columns 'Country Name' and 'GDP per Capita (£)' not found for 'Average GDP per Capita by Country' plot.")

```

Gradient Boosting model pipeline

```

# You can tune parameters like n_estimators, learning_rate, max_depth, etc.
gb_pipeline = Pipeline(steps=[('preprocessor', preprocessor),
                                ('classifier', GradientBoostingClassifier(n_estimators=100,
                                learning_rate=0.1, max_depth=3, random_state=42))])

```



```

# Train the Gradient Boosting model
print("Training Gradient Boosting Classifier...")
gb_pipeline.fit(X_train, y_train)
print("Training complete.")

# Make predictions on the test set
y_pred_gb = gb_pipeline.predict(X_test)

# Evaluate the Gradient Boosting model
print("\nGradient Boosting Classifier Model Evaluation:")
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_gb))
print("Classification Report:\n", classification_report(y_test, y_pred_gb))
print("Accuracy Score:", accuracy_score(y_test, y_pred_gb))

# Optional: Plot ROC Curve for Gradient Boosting (for multi-class, using OvR strategy)
# Requires predicted probabilities
if hasattr(gb_pipeline.named_steps['classifier'], 'predict_proba'):
    y_pred_proba_gb = gb_pipeline.predict_proba(X_test)

from sklearn.preprocessing import label_binarize
from itertools import cycle
import matplotlib.pyplot as plt
import numpy as np

# Binarize the output for multi-class ROC
if 'y_test' in locals() and 'y' in locals():
    n_classes = len(np.unique(y))
    y_test_bin = label_binarize(y_test, classes=np.unique(y))

```

```

plt.figure(figsize=(10, 8))

fpr_gb = dict()
tpr_gb = dict()
roc_auc_gb = dict()
for i in range(n_classes):
    fpr_gb[i], tpr_gb[i], _ = roc_curve(y_test_bin[:, i], y_pred_proba_gb[:, i])
    roc_auc_gb[i] = auc(fpr_gb[i], tpr_gb[i])

# Plot all ROC curves
colors = cycle(['aqua', 'darkorange', 'cornflowerblue', 'red', 'green', 'purple'])
for i, color in zip(range(n_classes), colors):
    plt.plot(fpr_gb[i], tpr_gb[i], color=color, lw=2,
             label='ROC curve of class {0} (area = {1:0.2f})'.format(np.unique(y)[i],
roc_auc_gb[i]))

plt.plot([0, 1], [0, 1], 'k--', lw=2)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve - Gradient Boosting')
plt.legend(loc="lower right")
plt.show()

else:
    print("Test data (y_test) or original target variable (y) not found. Cannot plot ROC
curve.")

else:
    print("Gradient Boosting classifier does not have predict_proba method. Cannot plot ROC
curve.")

```

```

# Get the trained model from the pipeline
gb_model = gb_pipeline.named_steps['classifier']

# Transform the test data using the same preprocessor
X_test_transformed = preprocessor.transform(X_test)

# Get feature names from the preprocessor
feature_names = preprocessor.get_feature_names_out()

# Initialize SHAP explainer for multi-class models
# Explain predictions for each class

explainer = shap.Explainer(gb_model.predict_proba, X_test_transformed,
feature_names=feature_names)

# Compute SHAP values
# shap_values is a list of arrays, one for each class
shap_values = explainer(X_test_transformed)

# Summary plot for the first class (you can change the index to visualize other classes)
class_index_to_visualize = 0
class_name_to_visualize = gb_model.classes_[class_index_to_visualize]

print(f"Generating SHAP summary plot for class: {class_name_to_visualize}")
shap.summary_plot(shap_values[:, :, class_index_to_visualize], X_test_transformed,
feature_names=feature_names, class_names=gb_model.classes_)

# Using KNN model to predict the attack type
if 'Attack Type' in df.columns:
    X = df.drop('Attack Type', axis=1)
    y = df['Attack Type']

```

```

# Identify numerical and categorical columns
numerical_features = X.select_dtypes(include=np.number).columns
categorical_features = X.select_dtypes(include='object').columns

# Create preprocessing pipelines
numerical_transformer = StandardScaler()
categorical_transformer = OneHotEncoder(handle_unknown='ignore')

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numerical_transformer, numerical_features),
        ('cat', categorical_transformer, categorical_features)
    ]
)

# Split the data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Fit and transform the training data
X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

# Create and train the KNN model
knn_pipeline = Pipeline(steps=[
    ('classifier', KNeighborsClassifier(n_neighbors=5))
])
knn_pipeline.fit(X_train_processed, y_train)

```

```

# Make predictions
y_pred_knn = knn_pipeline.predict(X_test_processed)

# Evaluate the KNN model
print("\nK-Nearest Neighbors (KNN) Model Evaluation:")
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_knn))
print("Classification Report:\n", classification_report(y_test, y_pred_knn))
print("Accuracy Score:", accuracy_score(y_test, y_pred_knn))

# Plot the confusion matrix
cm = confusion_matrix(y_test, y_pred_knn)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=np.unique(y), yticklabels=np.unique(y))
plt.title("Confusion Matrix - KNN")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.tight_layout()
plt.show()

else:
    print("\n'Attack Type' column not found. Cannot perform KNN classification without a
    target variable.")

# Final Performance Summary (Print this at the end of your notebook)
# ANSI escape code for bold
BOLD = "\033[1m"
END = "\033[0m"

print(f'{BOLD} Final Model Performance Summary{END}')
print(f'{BOLD} Gradient Boosting Classifier:{END}')

```

```
print(f'{BOLD} - Before Accuracy Score:{END} {accuracy_score(y_test, y_pred_gb):.4f}')
print(f'{BOLD} - Classification Report:{END}')
print(classification_report(y_test, y_pred_gb))
```

```
print(f'\n{BOLD} K-Nearest Neighbors (KNN):{END}')
print(f'{BOLD} - Accuracy Score:{END} {accuracy_score(y_test, y_pred_knn):.4f}')
print(f'{BOLD} - Classification Report:{END}')
print(classification_report(y_test, y_pred_knn))
```